

2

MINIMIZATION OF BOOLEAN FUNCTION

2.1. Boolean Algebra

Boolean algebra is a system of mathematical logic. It is an algebraic system consisting on the set of elements (0, 1) two binary operators called OR, AND and one unary operator NOT. It is the basic mathematical tools in the analysis and the synthesis of switching circuits. It is a way to express logic functions algebraically.

Note: Any functional relation in Boolean algebra can be provided by the method of perfect induction perfect inductions the method of proof, where by a function relation is verified for every possible combination of values that the value may assume.

Axioms of Boolean Algebra:

Axioms of Boolean algebra are a set of logical expressions that we accept without proof and upon which we can build a set of useful theorem.

	AND operator	OR operator	NOT operators
Axioms 1:	$0 \cdot 0 = 0$	$0 + 0 = 0$	$\bar{1} = 0$
Axioms 2:	$0 \cdot 1 = 0$	$0 + 1 = 1$	$\bar{0} = 1$
Axioms 3:	$1 \cdot 0 = 0$	$1 + 0 = 1$	
Axioms 4:	$1 \cdot 1 = 1$	$1 + 1 = 1$	

Logic operations: In Boolean Algebra all the algebraic function performed is logical. These actually represents logic operations. The AND, OR and NOT are the basic operations that are performed in Boolean Algebra. In addition to these operations, there are some derived operations such as NAND, NOR, EX-OR and EX-NOR that are also performed in Boolean Algebra.

1.1.1. NOT Operation

The NOT operation in Boolean algebra is similar to inversion in ordinary algebra

1 : $\bar{0} = 1$

2 : $\bar{1} = 0$

3 : if $A = 0$ then $\bar{A} = 1$

4 : $\bar{\bar{A}} = A$ (Double inversion)

1.1.2. AND Operation

It is a logical operation that are performed by AND gate. The AND operation in Boolean Algebra is similar to multiplication in ordinary algebra.

1: $A \cdot 0 = 0$ (Null Law)



2: $A \cdot 1 = A$ (Identity law)

3: $A \cdot A = A$

4: $A \cdot \bar{A} = 0$

1.1.3. OR Operation

It is the logical operation that are performed by OR gate. The OR operation in Boolean Algebra is similar to addition in ordinary algebra.

1: $A + 0 = A$ (Null law)

2: $A + 1 = 1$ (Identity law)

3: $A + A = A$

4: $A + \bar{A} = 1$

1.1.4. NAND Operation:

The NAND operation in Boolean Algebra is performed by AND operation followed by NOT operation i.e., the negation of AND operation is performed by NAND gate.

1.1.5. NOR Operation:

The NOR operation in Boolean Algebra is performed by OR operation followed by NOT operation i.e., the negation of OR operation is performed by NOR gate.

2.2. Laws of Boolean Algebra

2.2.1. Commutative Law

1. $A + B = B + A$

$$A + B + C = B + C + A = C + A + B = B + A + C$$

2. $AB = BA$

$$A \cdot BC = B \cdot CA = C \cdot AB = B \cdot AC$$

Violation: Inhibition (1) for Example x/y (x but not y) is not commutative law it means $x/y \neq y/x$

2.2.2. Associative law:

This law allows grouping of variables

1. $(A + B) + C = A + (B + C)$

$$A + (B + C + D) = (A + B + C) + D$$

$$= (A + B) + (C + D)$$

2. $(A \cdot B)C = A \cdot (B \cdot C)$

$$A(BCD) = (ABC) \cdot D$$

$$A(BCD) = AB \cdot CD$$

Note:- NAND and NOR gates are not Associative

2.2.3. Distributive Law

1: $A(B + C) = AB + AC$

$$A + BC = (A + B)(A + C)$$

2.2.4. Redundant Literal Rule

1. $A + \bar{A}B = A + B$
2. $A(\bar{A} + B) = AB$

2.2.5. Idempotent Law

1. $A \cdot A = A$
2. $A + A = A$

2.2.6. Absorption Law

1. $A + AB = A$
2. $A(A + B) = A$

2.2.7. Involutionary Law

The law that for any variable A.

$$\bar{\bar{A}} = (A')' = A$$

2.2.8. Consensus theorem:

There are two consensus theorems

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

$$(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$$

2.3. De-Morgan's theorem:

De-Morgan's theorem represents two of the most important rules of Boolean algebra.

$$\text{I. } \overline{A \cdot B} = \bar{A} + \bar{B}$$

$$\text{II. } \overline{A + B} = \bar{A} \cdot \bar{B}$$

The above two laws can be extended for 'n' variables as,

$$\overline{A_1 \cdot A_2 \cdot A_3 \dots + A_n} = \bar{A}_1 + \bar{A}_2 + \dots \bar{A}_n \text{ and } \overline{A_1 + A_2 + \dots + A_n} = \bar{A}_1 \cdot \bar{A}_2 \dots \bar{A}_n$$

2.3.1 Duality theorem:

Duality Theorem states that,

- (a) Change each OR sign by an AND sign and vice versa.
- (b) Complement any '0' or '1' appearing in expression
- (c) Keep literals as it is.

Note: With n variables, maximum possible distinct logic function = 2^{2^n}

Example : If a function is given as $f = AB + \bar{A}\bar{B}$ then find its complement.

Solution : Given $f = (AB + \bar{A}\bar{B})$

$$\begin{aligned}\text{Complement of } \bar{f} &= \overline{AB + \bar{A}\bar{B}} = \overline{AB} \cdot \overline{\bar{A}\bar{B}} \\ &= (\bar{A} + \bar{B})(A + B) \\ &= A\bar{A} + A\bar{B} + B\bar{A} + B\bar{B} = A\bar{B} + \bar{A}B\end{aligned}$$

Example : Show that

$$AB + B\bar{C} + AC = AC + B\bar{C}$$

Solution : LHS = $AB + B\bar{C} + AC$

$$\begin{aligned}&= AB(C + \bar{C}) + B\bar{C}(A + \bar{A}) + A(B + \bar{B})C \\ &= ABC + AB\bar{C} + ABC + \bar{A}B\bar{C} + ABC + A\bar{B}C \\ &= ABC + AB\bar{C} + \bar{A}B\bar{C} + A\bar{B}C \\ &= AC(B + \bar{B}) + B\bar{C}(A + \bar{A}) = AC + B\bar{C} \\ &= \text{RHS}\end{aligned}$$

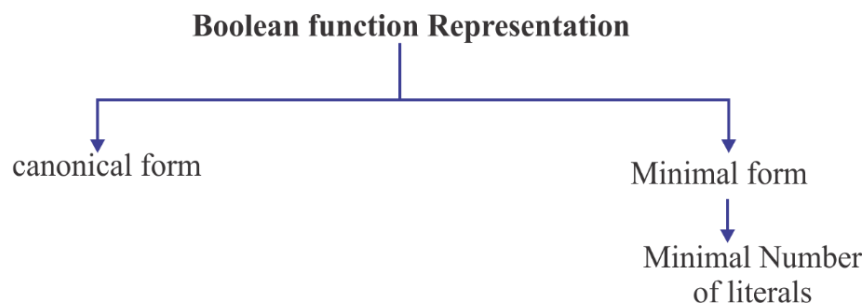
2.4. Minimization of Boolean function:

Every Boolean function expression must be reduced to as simple form as possible before realization because every logic operation in the expression represents a corresponding elements of hardware. Realization of digital circuit with minimal expression has several advantages as:

1. The number of logic gates will reduced.
2. The speed of operation will increase
3. power dissipation will decrease
4. The FAN IN may reduced
5. The complexing of the circuit reduces

The simple method of minimization of Boolean function using certain Algebraic rules which results in the reduction of number of term and/or number of arithmetic operations the various theorem and rules that are already discussed are very useful for the simplification of Boolean expression.

A function of n Boolean variables denoted by $f(A_1, A_2, \dots, A_n)$ is another variable of Algebra and takes one of the two possible values either 0 or 1. The various way of representing a given function are discussed below.



All the terms contain all the variable either in complementary or in uncomplimentary form

The literal means the Binary variable either in complementary or in uncomplimentary form.

2.4.1. Minimization of Boolean function using k-map

- **Using K-map:** The Boolean function can be simplified Algebraically but being not a symmetric method, we can never be sure that whether the minimal expression obtained is the real minimal or not.
- **Karnaugh Map (k-map):** A k-map is a graphical representation of Boolean expression, A two variable k-map will have four cell or squares 3-variable k-map will have 8-cells, n-variable k-map will have 2^n cells.

Note: Adjacent cells differ by 1 bit to maintain adjacently property gray code sequence is used in k-maps (Any two adjacent cells will differ by only one bit)

Min terms & Max terms:

1. n-binary variable have 2^n possible combinations.
 2. Min term is a product term, it contains all the variables either complementary or un complementary form for that combination the function output must be '1'.
 - Max term is a sum term, it contains all the variables either complementary or uncomplimentary form for that combination the function output must be '0'.
- For two variable

x	y	Min terms	Max terms
0	0	$m_0 = \bar{x} \bar{y}$	$M_0 = x + y$
0	1	$m_1 = \bar{x} y$	$M_1 = x + \bar{y}$
1	0	$m_2 = x \bar{y}$	$M_2 = \bar{x} + y$
1	1	$m_3 = x y$	$M_3 = \bar{x} + \bar{y}$

- In min terms we assigns '1' to each uncomplemented variables and '0' to each complemented variable.
- In Max terms we assign '0' to each uncomplemented variable and '1' to each complemented variables.

2.5. Representation of Boolean Functions

Any Boolean expression can be expressed in two forms

- Sum of Product form (SOP)
- Product of Sum form (POS)

2.5.1. SOP Form

The SOP expression usually takes the forms of two or more variables OR together.

$$Y = \bar{A}BC + A\bar{B} + AC$$

$$Y = A\bar{B} + B\bar{C}$$

SOP forms are used to write logical expression for the output becoming logic '1'.